First Steps with Your QNX System

Lecture 2

Overview

Learning Objectives

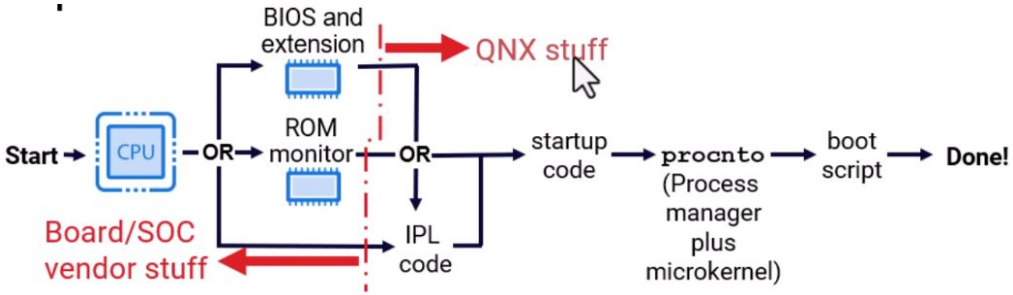
- Understand the QNX boot sequence and image structure
- Navigate the QNX shell and filesystem effectively
- File Ownership and Permissions
- Connect to your Raspberry Pi via WiFi and SSH
- Write, compile, and run your first QNX program
- Master essential QNX development workflow

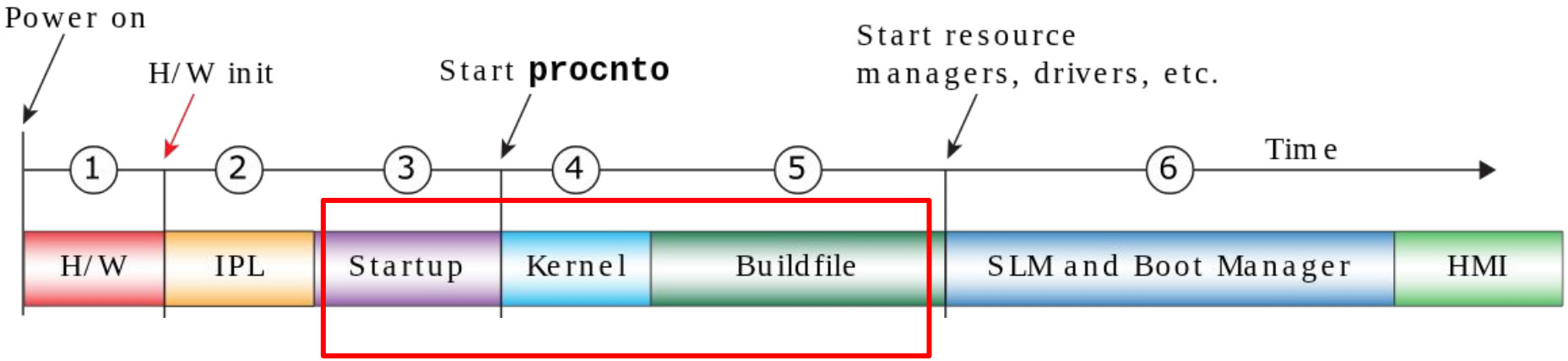
Topic 0 - QNX Booting Sequence

From Power-On to Full OS

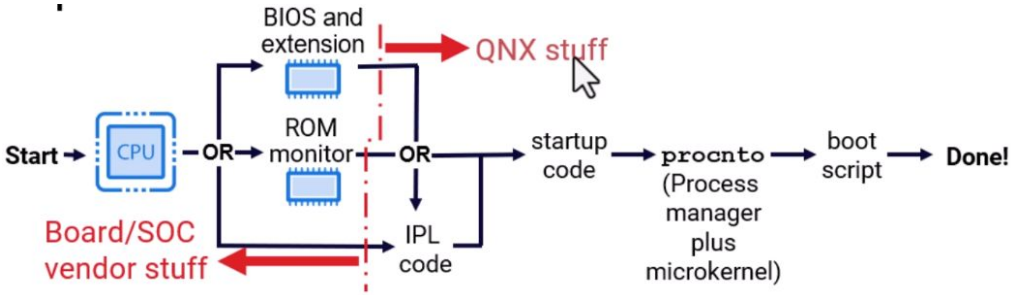
High Level Boot Sequence Overview

A useful way to think about the boot process and its components is that each step provides greater abstraction from the hardware. The IPL is specific to an architecture, board, and sometimes even board revision. The startup code is specific to an architecture and board. And, generally speaking, an OS kernel can work with different boards that share an architecture.





High Level Boot Sequence Overview



Step1: Before QNX — Provided by the board/SOC vendor

- **ROM Monitor** (like u-Boot, RedBoot, Bare-metal loader)
- **BIOS / UEFI** (in x86 systems)
- **On-chip ROM boot** (common in ARM)

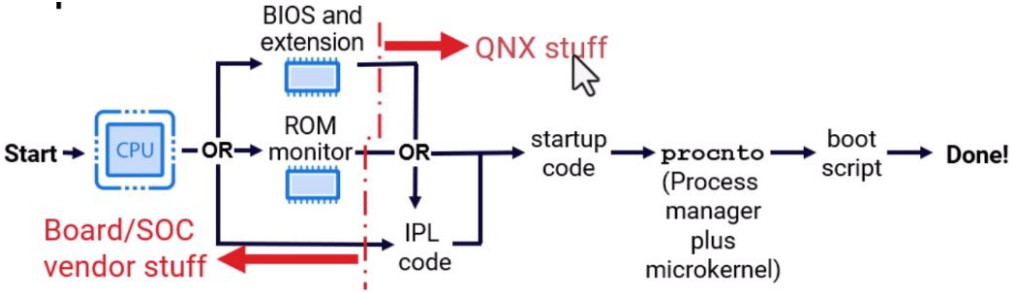
The board hardware and firmware perform checks and initialization tasks, then hand control over to the software located at a predetermined location.
Their job is to: load and execute the **IPL** (Initial Program Loader)

These components are *not* part of QNX.

High Level Boot Sequence Overview

QNX Boot Components

Step 2: IPL (Initial Program Loader)



The **IPL is the first QNX-specific code** that runs.

After finishing basic hardware preparation, it loads and locates the **IFS(Image File System)**

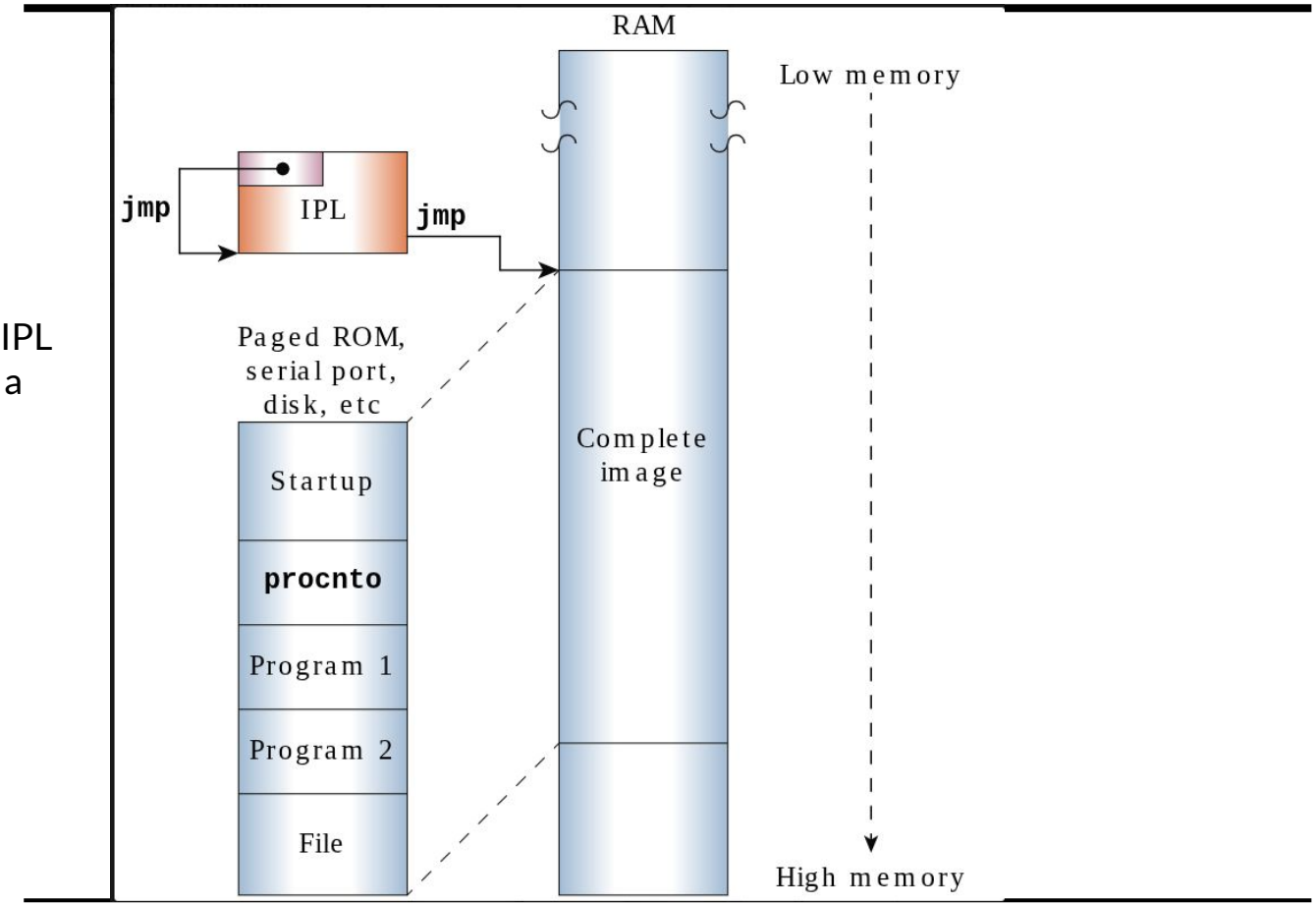
- Complete hardware initializations, if needed.
- Set up a stack for the C-language environment.
- Find and validate the OS image.
- Load the OS image into memory RAM.
- Jump to the startup code in the OS image.

The IPL moves the IFS from its storage location (like an SD card or Flash) into the RAM so that the startup program and QNX kernel can begin running

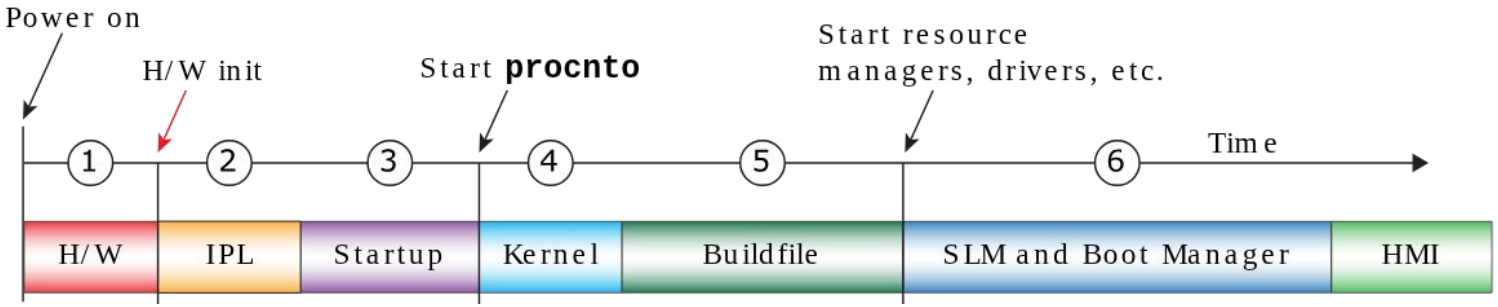
- Warm-start IPL
If there's a BIOS or ROM monitor already installed at the reset vector, then your IPL code is simply an extension to the BIOS or ROM monitor.
- Cold-start IPL
The system doesn't have (or doesn't use) a BIOS or ROM monitor program. The IPL must be located at the reset vector

QNX

Figure showing how a QNX IPL loads the entire image from a non-linearly mapped device into RAM.



High Level Boot Sequence Overview



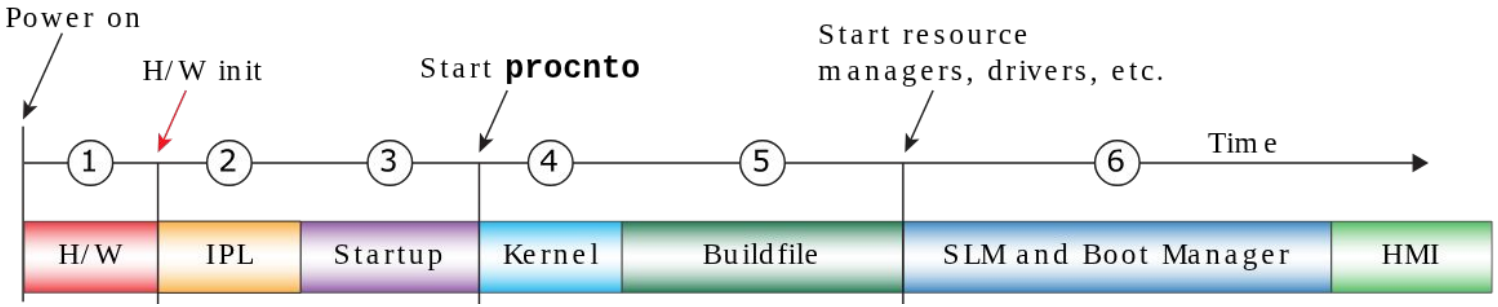
QNX Boot Components

Step 3 : **Startup (Startup Code) : This module takes over after the first-stage bootloader (BIOS, UEFI, U-boot, or IPL) has finished executing. The startup code executes after it has been copied from the boot device into the system's DRAM.**

The startup code's main tasks are, in sequence:

- The startup program configures the system, including timers, interrupt controllers, and cache controllers
- When it has completed the configuration, this program transfers control to the OS kernel.

High Level Boot Sequence Overview



QNX Boot Components

Step 4 : The Kernel : is included in the **procnto** module, which combines the kernel and process manager.

- **This module sets up the kernel, then runs the instructions specified in a script in the buildfile used to generate the OS image. This script contains startup instructions for drivers and other processes, and any additional commands for running other required components.**

High Level Boot Sequence Overview

QNX Boot Components

Step 5: Boot Script(sysinit): is a text file (script) that runs automatically right after the QNX kernel (procnto) finishes starting up.

Its main job is to launch all the necessary services and programs after boot.

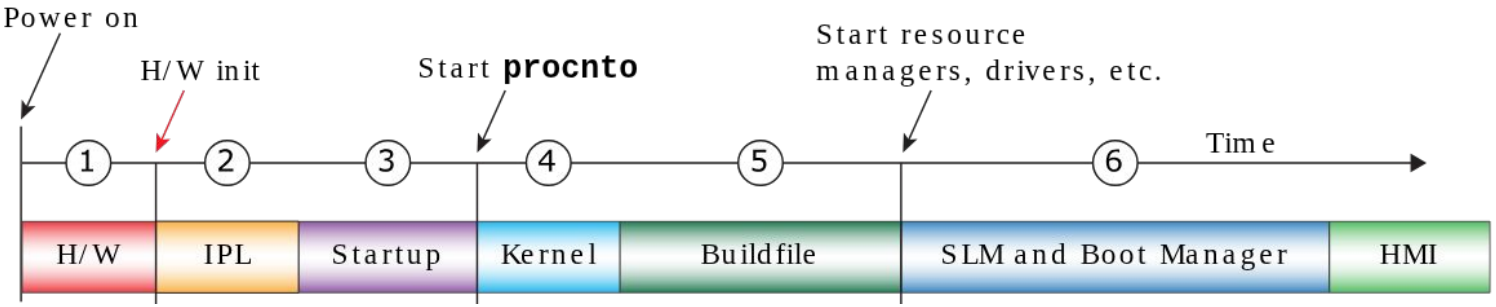
The boot script is where you add:

- the files and commands to include in the image
- the startup order for the executables
- the loading options for the files and the executables
- the command-line arguments and environment variables for the executables

a sequence of commands that you want the kernel and process manager to execute when it has completed its own startup

When all of the commands in the buildfile have completed successfully, the QNX system has finished booting.

High Level Boot Sequence Overview



QNX Boot Components

Step 6 : The System Launch and Monitor (SLM): utility is used to launch complex applications consisting of many processes that must be started in a specific order. It allows you to configure application launch sequences without modifying the buildfile

- If a component isn't required for the initial startup, you can take the component binary out of the IFS and manage its launch with the SLM service. This approach reduces the size of the IFS and makes easier to modify your startup sequence.

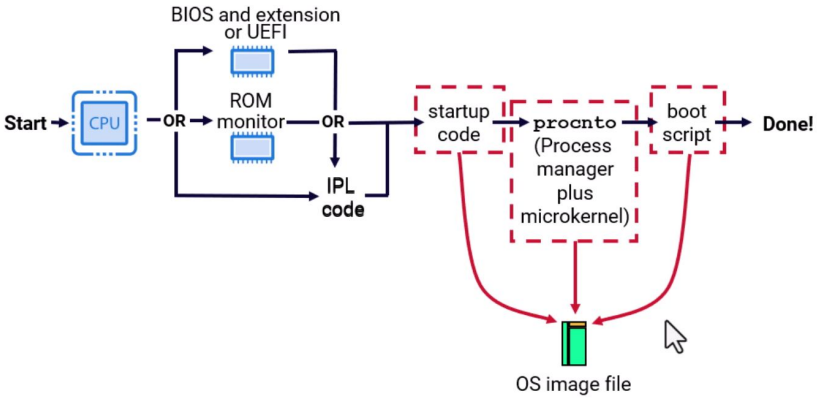
High Level Boot Sequence Overview

QNX Boot Components

IFS (Image Filesystem)

The IFS contains everything QNX needs to start running.

- **Startup code**
- **procnto** → The QNX Neutrino *microkernel* + process manager
- **Boot script** → *Very important*



The time needed to copy the IFS into RAM varies according to the IFS contents (and hence, size). It is usually the longest part of the kernel boot process.

High Level Boot Sequence Overview

QNX Boot Components

Image File Structure

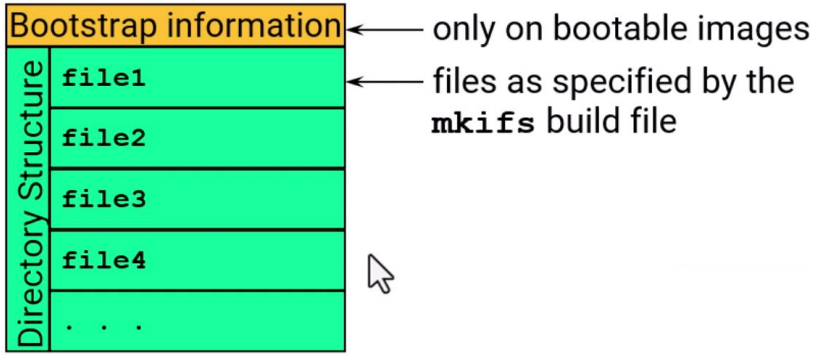
1 — 2

Bootstrap Section

Initial loader information at the front of the file

File System

Contains all files needed at boot time



Design Philosophy

Minimalist: Only essential drivers
Expandable: Load more from flash/SD after boot
Flexible: Can include everything OR just basics
Board-Specific: Tailored to hardware capabilities

Note: Use **dumpifs** to view .ifs file content.

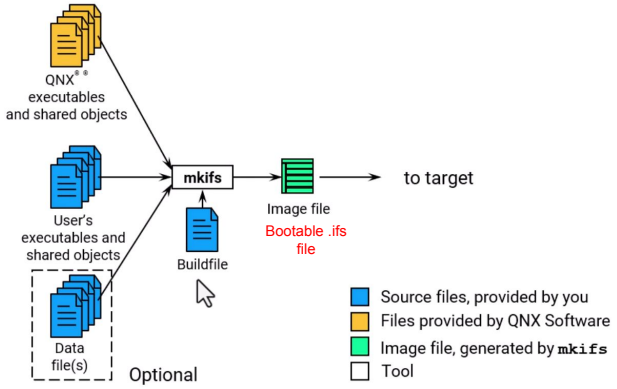
mkifs and the Buildfile

QNX Boot Components

mkifs: Make Image Filesystem

What is a Buildfile?

mkifs uses rules from the buildfile to combine OS and user executables, shared objects and data files to create an OS image (IFS), which can be transferred to the target system and booted.




```
mkifs [-?] [-a suffix] [-l inputline] [-n[n]] [-o directory]
      [-p patchfile] [-r rootdir] [-s section] [-v]
      [buildfile] [directory] [outputfile]
```

mkifs and the Buildfile

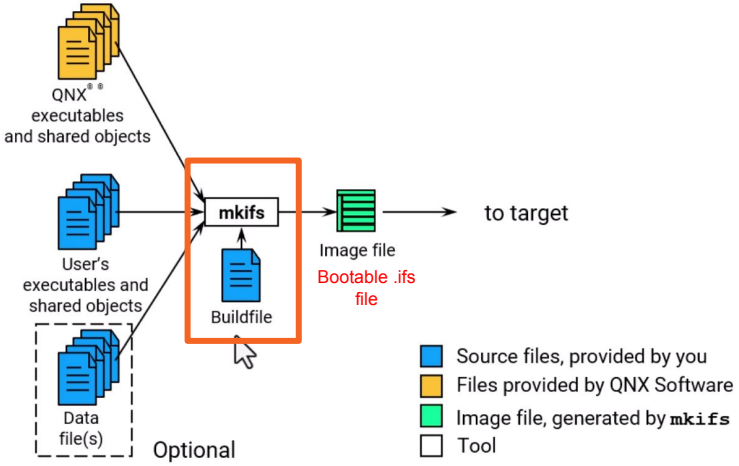
QNX Boot Components

Buildfile

What is a Buildfile?

- a text file that specifies:

- files/commands that will be included in the image
- loading options for files & executables.
- options for creating the image (e.g. compression)
- for bootable images, a boot script that says what to run after OS initialization



What appears in /proc/boot

Once booted, everything you put in the image file appears in /proc/boot by default. This is a read-only, in-memory filesystem. It is not on any disk — it is the image itself, mapped into RAM. You can redirect files to appear at other paths (like /etc instead of /proc/boot) using path assignments in the buildfile.

QNX

QNX Boot Components

Buildfile Essential Contents

mkifs and the Buildfile

- bootstrap loader:
  - startup-boardname
- and operating system:
  - procnto-smp-instr
- boot script
- executables and shared libraries
  - executables aren't strictly required, but then the system wouldn't actually ***do*** anything without them!
  - but, in most cases, to run any executable, you need at least ***libc.so, libgcc_s.so, and ldqnx-64.so***
  - and most QNX supplied services also require ***libsecpol.so***

target PROCESSOR

```
#     This is "hello.build"
[virtual=x86_64,multiboot] .bootstrap = {
    startup-x86
    PATH=/proc/boot procnto-smp-instr
}
[+script] .script = {
    procmgr_symlink /proc/boot/ldqnx-64.so.2 /usr/lib/ldqnx-64.so.2
    devc-ser8250 -e -b115200 &
    reopen /dev/ser1
    hello
}
libc.so
libgcc_s.so.1
ldqnx-64.so.2
libsecpol.so
devc-ser8250
hello
```

bootstrap file

startup script

shared libraries

executables

Explore the QNX World

Additional Resources

- QNX Boot Process
- QNX Neutrino FileSystems
- QNX Neutrino Microkernel - Process and Threads
- QNX 8.0 OS User Guide - Process and Threads
- QNX Meeting Early Boot Requirements (Advanced)
- SYMMETRIC MULTIPROCESSING (SMP) AT A GLANCE (Advanced)

Thank You

We will continue ....